Module 12: Events, Listeners & Notifications

Duration: 4-5 days | **Level:** Intermediate-Advanced

Events & Listeners

Decouple actions from reactions:

```
// Event
class TaskCompleted
{
use Dispatchable, SerializesModels;
public function __construct(public Task $task, public User $completedBy) {}
}

// Listener
class LogTaskCompletion
{
public function handle(TaskCompleted $event): void
{
ActivityLog::record('task.completed', $event->task, $event->completedBy);
}
}

// Dispatch
event(new TaskCompleted($task, $user));
```

Register in `AppServiceProvider`:

```
Event::listen(TaskCompleted::class, LogTaskCompletion::class);
```

Model Observers

```
php artisan make:observer TaskObserver --model=Task

class TaskObserver
{
public function updated(Task $task): void
{
if ($task->wasChanged('status')) {
// React to status change
}
}
}
```

Notifications

Multi-channel alerts (mail, database, Slack, SMS):

```
class TaskDueReminder extends Notification implements ShouldQueue
{
public function via(object $notifiable): array
{
return ['mail', 'database'];
}

public function toMail(object $notifiable): MailMessage { ... }
public function toArray(object $notifiable): array { ... }
}
```

Send:

```
$user->notify(new TaskDueReminder($task));
Notification::send($users, new TaskDueReminder($task));
```

Broadcasting (Real-time)

For WebSockets with Laravel Echo + Pusher/Ably:

```
composer require pusher/pusher-php-server
php artisan install:broadcasting
```

Exercises

1. Create TaskAssigned event that emails the assignee.
2. Add database notifications table: php artisan notifications:table.
3. Display unread notifications in the TaskForge navbar.

Next Module

-> Module 13: Caching & Performance